

MLOps – session2 :Shipping to Production

السيشن ديه مخصصة لتعليمك كيفية أتمتة وبناء pipeline متكامل لنموذج الذكاء الاصطناعي (Automated Pipeline). سنتحدث في هذه السيشن عن 7 مواضيع أساسية :

1. **MLflow** : لتعقب التجارب وإدارة النماذج.
2. **Weights & Biases** : كبديل سحابي لإدارة التجارب.
3. **DVC (Data Version Control)** : لإدارة نسخ البيانات والنماذج.
4. **Code Quality** : لضمان جودة الكود باستخدام أدوات الفحص التلقائي.
5. **Terraform** : لإدارة البنية التحتية ككود. (Infrastructure as Code).
6. **GitHub Actions** : لأتمتة عمليات الدمج والنشر المستمر (CI/CD).
7. **Integration Testing** : لاختبار النظام بأكمله كدورة كاملة.

أولاً: ماذا بنينا في Session 1؟ (Context Recap)

قبل البدء بأتمتة كل شيء، سنفترض أنك تمتلك بالفعل نظاماً أساسياً قمت ببنائه سابقاً ويحتوي على:

- **Python package** : كود منظم داخل مجلد `src/` ومعرّف بواسطة ملف `pyproject.toml`.
- **Litestar API** : واجهة برمجية (API) تحتوي على مسارات مثل `/predict` و `/health` باستخدام مكتبة `Pydantic` للتحقق من البيانات.
- **Docker** : ملفات تشغيل الحاويات باستخدام `Multi-stage builds` وأداة `docker-compose`.
- **pytest suite** : اختبارات تغطي على الأقل 80% من الكود وتستخدم `TestClient`.
- **Structured logs** : سجلات منظمة بصيغة **JSON** تحتوي على `correlation ids` لتتبع الطلبات.
- **README** : ملف توثيقي يشرح كيفية تشغيل المشروع بثلاثة أوامر فقط.

MLflow (Never lose a good experiment again)

1. لماذا نحتاج MLflow؟ (Why MLflow)

بدون استخدام MLflow ، يعيش مهندس الماشين ليرنينج في فوضى عارمة عند تدريب النماذج. ستجد في مجلداتك ملفات مثل:

• model_v1_final.pkl

• model_v1_final_FINAL.pkl

• model_v2_fixed_real.pkl

وتقع حينها في حيرة: أي نموذج حقق دقة (Accuracy) أعلى؟ وما هي الـ Hyperparameters الدقيقة التي استخدمناها لإنتاج هذا النموذج؟

باستخدام MLflow ، يتم استبدال هذه الفوضى بنظام منظم يعتمد على أربعة مفاهيم رئيسية:

• **Experiment** : مجلد أو تصنيف يحمل اسماً معيناً ويجمع بداخله جميع محاولات التدريب ذات الصلة بالمشكلة.

• **Run** : عملية تنفيذ وتدريب واحدة مسجل فيها كافة الـ Params والـ Metrics الخاصة بها.

• **Artifact** : الملفات المرفقة بالـ Run (مثل ملفات النموذج نفسه كـ .pkl أو الرسومات التوضيحية والبيانات).

• **Registry** : مستودع مركزي للنماذج يوفر إمكانية التحكم في إصداراتها ودورة حياتها من مرحلة **Staging** إلى مرحلة **Production** .

2. إعداد الاتصال بالسيرفر (MLflow Setup)

لتشغيل MLflow ، نقوم بربط الكود بالسيرفر الخاص بالتعقب بسطرين فقط قبل تشغيل أي كود تدريب:

```
import mlflow

# تحديد عنوان السيرفر (Tracking URI)
mlflow.set_tracking_uri("http://localhost:5000")

# تحديد اسم التجربة لربط كل عمليات التدريب القادمة بها
mlflow.set_experiment("ride-duration-model")
```

- **Tracking URI** : يمثل مكان حفظ البيانات. في مرحلة التطوير نشير إلى localhost وفي بيئة العمل CI نشير إلى سيرفر الفريق.
- **Experiment** : سلة تجميع لل-Runs ؛ ننشئ تجربة واحدة لكل مشكلة وليس لكل محاولة.
- **Containerized** : يعمل MLflow ك-service منفصلة بجانب ال-API الخاصة بك داخل نفس ملف docker-compose.yml

3. تسجيل البيانات أثناء التدريب (MLflow Tracking)

نسجل البارامترات وال-metrics والنموذج باستخدام الكود التالي:

```
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

# حتى لو تعطل الكود أثناء التدريب Run لضمان إغلاق ال-Context Manager استخدام
with mlflow.start_run(run_name="rf-baseline") as run:

    params = {"n_estimators": 100, "max_depth": 6}
    mlflow.log_params(params) # تسجيل ال-Hyperparameters

    model = RandomForestRegressor(**params)
    model.fit(X_train, y_train)

    mae = mean_absolute_error(y_val, model.predict(X_val))
    mlflow.log_metric("mae", mae) # تسجيل المقاييس

# مباشرة Registry تسجيل النموذج في ال-
mlflow.sklearn.log_model(
    model, artifact_path="model",
    registered_model_name="RideDurationModel")
```

- **start_run()** : تضمن إغلاق السيرفر وحفظ حالته حتى لو حدث Crash للبرنامج.
- **log_params()** : ترسل البارامترات في شكل قاموس (Dictionary) ليسهل مقارنتها لاحقاً في واجهة المستخدم (UI).
- **log_metric()** : تسجل الأرقام الفردية، ويمكن استدعاؤها مراراً لتسجيل منحنيات التدريب (Training Curves).
- **log_model()** : تحفظ أوزان النموذج والبيانات الوصفية (Metadata). وإضافة registered_model_name تدفع بالنموذج مباشرة إلى ال-Registry.

4. مقارنة النتائج (Run Comparison)

هنا تظهر القوة الحقيقية لـ MLflow . فبدلاً من مراجعة عشرات الملفات البرمجية، تتيح لك واجهة المستخدم فرز ومقارنة النماذج المختلفة في جدول مرتب بحسب مقياس الخطأ مثل الـ MAE لتحديد النموذج الأفضل بسهولة (مثل مقارنة نماذج Linear Regression و XGBoost و PyTorch).

5. إدارة النماذج (Model Registry)

بعد استقرارنا على النموذج الأفضل، نستخدم الـ Registry لنقله عبر المراحل المختلفة في دورة حياته دون الحاجة لتغيير كود الخدمة (Serving Code) :

```
from mlflow import MlflowClient

client = MlflowClient("http://localhost:5000")

# Staging البحث عن إصدارات النموذج وترقيتها إلى مرحلة
client.transition_model_version_stage(
    name="RideDurationModel",
    version="3", stage="Staging")

# وليس رقم النسخة يدوياً (Stage) كود الخدمة يطلب النموذج عبر المرحلة
model =
mlflow.sklearn.load_model("models:/RideDurationModel/Production")
```

الـ stages الأربعة المتاحة هي:

1. None : تم تسجيله حديثاً.
2. Staging : تحت عملية التحقق والتجربة.
3. Production : يخدم طلبات المستخدمين الحية (Live Traffic) .
4. Archived : نموذج قديم تم حفظه في الأرشيف ومتقاعد.

عن أبو هريرة رضي الله عنه قال - قال رسول الله صلى الله عليه وسلم : إِنَّ الْعَبْدَ لَيَتَكَلَّمُ بِالْكَلِمَةِ مِنْ رِضْوَانِ اللَّهِ، لَا يُلْقِي لَهَا بَالًا، يَرْفَعُهُ اللَّهُ بِهَا دَرَجَاتٍ، وَإِنَّ الْعَبْدَ لَيَتَكَلَّمُ بِالْكَلِمَةِ مِنْ سَخَطِ اللَّهِ، لَا يُلْقِي لَهَا بَالًا، يَهْوِي بِهَا فِي جَهَنَّمَ. (رواه البخاري)

6. ميزات MLflow الستة الأساسية (Feature Map)

1. **Experiment tracking** : مقارنة تشغيلين جنباً إلى جنب في الـ UI عبر تسجيل المقاييس والبارامترات والملفات.
2. **Autologging** : تفعيل سطر واحد يسجل تلقائياً كافة الـ parameters والـ metrics والرسومات للعديد من المكتبات الشهيرة مثل PyTorch و XGBoost و scikit-learn .
3. **Artifact logging** : حفظ ملفات إضافية مثل الـ (Confusion Matrices) والـ (Feature Importance) وصور SHAP .
4. **Model registry** : مخزن لإصدارات النماذج يتيح تحميلها بالاسم الحركي للمرحلة (Stage) بدلاً من كتابة رقم الإصدار يدوياً.
5. **Run comparison UI** : جدول تفاعلي ورسومات بيانية للمقارنة السريعة بين كافة الـ Runs.
6. **Tags and search** : Tag الـ Runs بمعلومات إضافية (مثل اسم الفريق، إصدار كود Git) مع إمكانية البحث باستخدام لغة شبيهة بـ SQL .

7. مسار LLM / RAG (LLM / RAG Track)

إذا كنت تعمل على مشاريع الـ Generative AI أو الـ RAG ، فإن MLflow يوفر نفس المفهوم ولكن بمدخلات ومخرجات مختلفة تناسب طبيعة هذه المهام:

المفهوم في الـ Classical ML	المقابل في الـ LLM / RAG	سبب الأهمية
Hyperparameters	Prompt version, chunk_size, chunk_overlap, temperature	دقة وجودة الاسترجاع تتأثر بحجم الـ Chunks، وكل تغيير في الـ Prompt بمثابة نموذج جديد.
Training metrics	RAGAS faithfulness, answer relevancy, context recall	هذه هي مقاييس التقييم الأساسية الخاصة بك؛ لا توجد حلقة تدريب تقليدية لمراقبتها.
Model artifact	System prompt text, embedding model, RAG config bundle	الـ Prompt هو الـ Artifact الأساسي في النظام، ويتم حفظ نسخته كأنها نموذج.

كود الخدمة يقوم بتحميل الـ Prompt بناءً على حالته ومرحلته في المستودع.	Prompt registry (Draft -> Staging -> Production)	Model registry
نفس واجهة المقارنة لترتيب وتصفية النتائج بناءً على مقاييس الـ RAGAS لتقييم الـ LLM.	Prompt A vs prompt B vs Mistral 7B vs Llama 3.1	Run comparison

(ملاحظة: بدءاً من الإصدار MLflow 2.12 ، تتوفر ميزات مخصصة للـ Prompt Registry مثل `register_prompt()` و `set_prompt_alias()`)

استخدام DVC لإصدار وإدارة بيانات الـ LLM / RAG

لكي تكون تجارب الـ LLM قابلة لإعادة الإنتاج (Reproducible) بنسبة 100%، يجب أن نقوم بإصدار البيانات غير المنظمة (Unstructured Data) التي يتغذى عليها النظام. وهنا يأتي دور **DVC** لإصدار ثلاثة أجزاء رئيسية:

1. الـ Knowledge base (قاعدة المعرفة) :

- في الـ ML التقليدي هذا هو ملف التدريب. أما هنا، فهو مجلد المستندات الخام (مثل ملفات الـ PDFs القانونية، وعقود الشركة، والكتيبات).
- نقوم بتتبعه ببساطة عبر الأمر :

```
dvc add data/knowledge_base/
```

2. الـ Chunk store (مخزن النصوص المجزأة):

- يمثل مرحلة الـ Feature Engineering. وهو ملف مثل `chunks.jsonl` الذي ينتج بعد عملية تنظيف النصوص وتجزئتها وحقن البيانات الوصفية (Metadata).
- يتم تشغيله وإعادة إنتاجه تلقائياً عند تغيير كود التجزئة أو الملفات الخام عبر الأمر :

```
dvc repro chunk_docs
```

3. الـ Eval set (مجموعة التقييم) :

- بمثابة الـ Test Labels في الـ ML التقليدي. وهي عبارة عن مجموعة ذهبية (Golden Dataset) تحتوي على 50 إلى 100 زوج من الأسئلة، الإجابات المرجعية (Reference Answers)، والمستندات المرتبطة بها.
- نقوم بتتبعها لضمان ثبات التقييم :

```
dvc add data/eval/test_set.jsonl
```

دورة العمل الفلاحية: إذا صدر قانون جديد، نقوم فقط بإضافة المستند الجديد للمجلد، ثم تشغيل dvc add و git commit، ثم يقوم dvc repro تلقائياً بإعادة بناء كشاف المتجهات (Vector Index) ليكون النظام محدثاً وموثوقاً.

ما الذي يجب إصداره وتتبعه أيضاً؟ (Versioning beyond files)

هناك عناصر برمجية متطورة وحرحة داخل نظام الـ LLM تحتاج أيضاً إلى إصدار وتتبع مستمر لضمان عدم حدوث أي انهيار للنظام:

- **The vector index** : وهو يدمج الـ Chunks مع الـ Embedding Model وبارامترات الـ Indices نفسه.
- **Embedding model version** : إذا قمت بتغيير نموذج الـ Embedding (مثلاً الانتقال من الإصدار v1 إلى v2)، فإن الـ vector space يتغير بالكامل ولا يمكن خلط الـ vectors القديمة بالجديدة.
- **Fine-tuning data** : بيانات التدريب المخصصة (مثل instruction pairs أو LoRA adapters).
- **Tool schemas and guardrails** : إذا تغير الـ Signature الخاص بالأدوات المتاحة للـ Agent، فإن سلوكه يتغير بالكامل.
- **The judge config** : في أنظمة التقييم، نستخدم نموذج LLM آخر كحكم (Judge). كود هذا الحكم والـ Prompt الخاص بالتقييم يعتبران جزءاً من المقياس ويجب إصدارهما.

متى لا يصلح استخدام DVC؟ وما هي البدائل؟ (When DVC does not fit)

الـ DVC ممتاز للملفات التي يمكنك حصرها وتعدادها يدوياً (Enumerated Files)، ولكن في الأنظمة الحية التي تكتب وتستقبل البيانات باستمرار، نلجأ للحلول التالية:

- **Vector DB snapshots** : نأخذ لقطة كاملة من قاعدة بيانات المتجهات (مثلاً بناء docs_v3) ثم نقوم بتبديل الـ Alias برمجياً بشكل لحظي ودون توقف للنظام.
- **Database PITR (Point-in-Time Recovery)** : نأخذ نسخة احتياطية من قاعدة البيانات الكلاسيكية (مثل pg_dump في PostgreSQL) ونربطها بالـ Git SHA.

- **Table time travel** : استخدام تقنيات مثل **Delta Lake** أو **Iceberg** للرجوع بالزمن وقراءة البيانات كما كانت في تاريخ معين.
- **lakeFS or Nessie** : تتيح لك عمل Branches و Merging على مستوى بحيرة البيانات بأكملها (Data Lake) كأنك تتعامل مع Git .

• نيجي للكود :

تم تصميم الكود بشكل كتل برمجية منفصلة (**Modular OOP structure**) لتسهيل صيانتها واختباره وتوسيع نطاقه مستقبلاً:

1. معالج تقطيع النصوص (Document Chunking)

- الكلاس **DocumentChunker** :
- دوره: يقوم باستقبال المستندات الضخمة وتقسيمها إلى أجزاء أصغر (**Chunks**) بناءً على حجم ثابت (**chunk_size**) مع الحفاظ على تداخل للنصوص (**chunk_overlap**) لمنع فقدان المعنى السياقي بين القطع المتتالية.
- الربط مع الـ **MLOps** : يتم تمرير حجم القطعة وتداخلها كمعاملات متغيرة (**Hyperparameters**) يتم تتبعها لاحقاً في **MLflow** لمقارنة تأثيرها على جودة الاسترجاع.

2. الـ (Vector Index & Similarity Search)

- الكلاس **VectorIndex** بالاستعانة بـ **SimpleEmbedder**
- دوره: في الأنظمة الحية، نقوم بربط هذا الجزء بنموذج خارجي (مثل **OpenAI** أو **SentenceTransformers**) وقاعدة بيانات متجهات (مثل **ChromaDB** أو **Pinecone**).
- التنفيذ الفعلي في الكود: لضمان عمل الكود لديك محلياً دون الحاجة لمفاتيح برمجية (**API Keys**) أو سحب مكتبات ضخمة، تم بناء محرك حساب تشابه جيب التمام (**Cosine Similarity**) رياضياً بالكامل باستخدام مكتبة **NumPy** لمقارنة متجهات النصوص واسترجاع أكثر جزأين متشابهين مع سؤال المستخدم (**top_k = 2**) .

3. محاكي نموذج اللغة الكبير (Mock LLM Engine)

- الكلاس **MockLLM** :

• دوره: يستقبل الـ **Prompt** النهائي الذي يحتوي على السياق المسترجع وسؤال المستخدم، ثم يقوم بصياغة الإجابة بناءً على درجة الإبداع المحددة له (**temperature**) .

4. المايسترو وتتبع التجارب (The RAG System & MLflow Tracking)

• الكلاس RAGSystem :

• دوره: هو العقل المدبر الذي يربط التقطيع والاسترجاع والـ LLM معاً.

◦ آلية التتبع بداخل MLflow :

- تتبع البارامترات (log_params) : نسجل حجم التجزئة وتداخلها، و Temperature والـ LLM ، والـ top_k المسترجعة.
- تتبع مقاييس الـ RAGAS (log_metrics) : نسجل محاكاة لمقاييس التقييم الثلاثة للـ RAG لضمان جودة الأداء وعدم الهلوسة.
- تتبع وحفظ الـ Artifacts (log_artifact) : يتم حفظ ملف JSON يحتوي على الـ Prompt المستخدم بالكامل، السياق المستخرج، ونسب التشابه لكي يستطيع أي مهندس في الفريق مراجعة وتدقيق عمل الـ LLM لاحقاً بالتفصيل.

هيكل الكود الأساسي المكتوب داخل rag-pipeline.py

```
# كود التتبع الرئيسي بداخل كلاس الـ RAGSystem
with mlflow.start_run(run_name="rag-query-run") as run:
    # 1. تسجيل بارامترات النظام (Hyperparameters)
    mlflow.log_params({
        "chunk_size": self.chunk_size,
        "chunk_overlap": self.chunk_overlap,
        "llm_temperature": self.temperature,
        "top_k_retrieved": 2
    })

    # 2. تسجيل مقاييس التقييم الحيوية (RAGAS Metrics)
    mlflow.log_metrics({
        "ragas_faithfulness": 0.95,
        "ragas_answer_relevancy": 0.91,
        "ragas_context_recall": 0.88
    })

    # 3. Artifacts حفظ مستندات التدقيق كملفات
    mlflow.log_artifact("scratch/rag_run_details.json", artifact_path="rag_payload")
```

كيف تقوم بتشغيل وتجربة هذا الكود؟

1. تأكد من تثبيت المكتبات الأساسية في بيئتك البرمجية :

```
pip install numpy mlflow
```

- قال تعالى: ﴿إِنَّ اللَّهَ وَمَلَائِكَتَهُ يُصَلُّونَ عَلَى النَّبِيِّ يَا أَيُّهَا الَّذِينَ آمَنُوا صَلُّوا عَلَيْهِ وَسَلِّمُوا تَسْلِيمًا﴾ [الأحزاب: 56]

2. لتشغيل سيرفر MLflow محلياً لمراقبة وتسجيل كافة تفاصيل تشغيل الـ RAG:

```
mlflow server --host 127.0.0.1 --port 5000
```

3. قم بتشغيل الاسكريبت البرمجي :

```
python rag-pipeline.py
```

4. افتح المتصفح على الرابط <http://localhost:5000> لمشاهدة واجهة المستخدم الرائعة لـ MLflow ، ومقارنة نتائج الـ Runs المختلفة وتفاصيل النصوص المسترجعة لكل تجربة بدقة متناهية!

تعالاً ندخل علي جزء مختلف وهو **Weights & Biases (W&B)**، وهو البديل السحابي القوي لـ MLflow سنتعلم كيف يعمل، وما الفارق بينه وبين MLflow ، وكيف نكتب كود التتبع الخاص به، بالإضافة إلى ميزته القاتلة: **Hyperparameter Sweeps** .

1. المقارنة بين MLflow vs. Weights & Biases :

على الرغم من أن الأدوات تؤديان نفس الوظيفة تقريباً (تتبع التجارب وإصدار النماذج)، إلا أن فلسفة البنية التحتية لكل منهما مختلفة تماماً:

وجه المقارنة	MLflow	Weights & Biases (W&B)
الاستضافة (Hosting)	Self-hosted : تقوم باستضافته وإدارته بنفسك على السيرفر الخاص بك.	Cloud SaaS : منصة سحابية جاهزة مستضافة على موقع wandb.ai .
مجهود الإعداد (Setup)	يستغرق حوالي 10 دقائق من خلال ضبط ملف <code>docker-compose.yml</code>	سريع جداً؛ فقط <code>pip install wandb</code> ثم <code>wandb login</code>
البحث عن المعاملات (Hyperparameter Search)	يتم يدوياً، أو تحتاج لدمجه مع مكتبة خارجية مثل Optuna .	مدمج به نظام Sweeps ذكي وتلقائي (يدعم البحث البايزي والسير العشوائي).
العمل الجماعي (Collaboration)	يعتمد على مشاركة سيرفر MLflow مركزي بين الفريق.	يوفر واجهات متقدمة لإدارة المشاريع، الفرق، وكتابة التقارير التفاعلية.

- لَأَنْ أَقُولَ : سُبْحَانَ اللَّهِ وَالْحَمْدُ لِلَّهِ وَلَا إِلَهَ إِلَّا اللَّهُ وَاللَّهُ أَكْبَرُ أَحَبُّ إِلَيَّ مِمَّا طَلَعَتِ عَلَيْهِ الشَّمْسُ

يتم تخزين الملفات محلياً أو على S3 الخاص بك.	مخزن الملفات (Artifact Storage)
يوفر باقة مجانية للأفراد، وباقات مدفوعة للفرق والشركات.	التكلفة (Cost)
التجارب السريعة، الأبحاث الأكاديمية، والفرق التي تفضل التقارير السريعة.	أفضل استخدام لـ (Best for)

في هذا الكورس، سنتعلم **MLflow** بعمق لأنه الأنسب للبيئات المحلية للشركات، ولكن يجب أن تعرف **W&B** جيداً وتستخدمه في الأبحاث وكتابة تقارير الفرق السريعة.

2. كتابة كود التتبع باستخدام W&B (Logging a Run to the Cloud)

بدلاً من واجهة **MLflow** المحلية، عند تشغيل هذا الكود، ستظهر لك النتائج والرسومات البيانية مباشرة على موقع **W&B** السحابي:

```
import wandb
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error

# 1. على السحابة وتحديد اسم المشروع والتجربة Run تهيئة الـ
run = wandb.init(project="ride-duration", name="rf-baseline")

# 2. (Immutable Config) تسجيل معاملات المدخلات
params = {"n_estimators": 100, "max_depth": 6}
wandb.config.update(params)

# تدريب النموذج التقليدي
model = RandomForestRegressor(**params)
model.fit(X_train, y_train)
mae = mean_absolute_error(y_val, model.predict(X_val))

# 3. (Metrics) تسجيل المقاييس
wandb.log({"mae": mae, "val_size": len(y_val)})

# 4. وإرساله للسحابة Artifact حفظ النموذج كـ
artifact = wandb.Artifact("ride-model", type="model")
artifact.add_file("model.pkl")
run.log_artifact(artifact)

# Run إنهاء الجلسة وإغلاق الـ
run.finish()
```

شرح الكود البرمجي لـ W&B :

- **wandb.init()** : تنشئ التجربة (Run) مباشرة على خوادم W&B وتمنحك كائن الـ run للتحكم في الـ Artifacts .
- **wandb.config** : مخزن ثابت لتسجيل برامترات التدريب، إصدارات البيانات، ونوع النموذج، مما يضمن ثبات المدخلات وعدم تعديلها.
- **wandb.log()** : تستخدم لتسجيل أي مقياس في أي خطوة. عند استدعائها داخل حلقات التدريب للشبكات العصبية (Neural Networks) ، ترسم لك منحنيات تدريب حية وتفاعلية لحظة بلحظة على المتصفح.
- **Artifact** : مستودع لحفظ الملفات والبيانات والـ Models مع تتبع إصداراتها تلقائياً لتصبح بمثابة "كائنات من الدرجة الأولى" يمكن لأي عضو في الفريق تحميلها وإعادة استخدامها.

3. الميزة القاتلة Sweeps : (Hyperparameter Search, Built-In)

في MLflow ، تحتاج لكتابة كود معقد لدمج مكتبة **Optuna** من أجل البحث عن أفضل توليفة للـ Hyperparameters . أما في W&B ، الميزة مدمجة بالكامل وتسمى **Sweeps** .

كل ما عليك فعله هو تعريف فضاء البحث وتشغيل الـ Agent :

```
# تعريف إعدادات البحث وقضائه 1.
sweep_config = {
    "method": "bayes", # استخدام البحث البايزي ذو الكفاءة العالية
    "metric": {"name": "mae", "goal": "minimize"}, # هدفنا تقليل مقياس الخطأ MAE
    "parameters": {
        "n_estimators": {"values": []}, # تجربة هذه القيم الثلاث
        "max_depth": {"min": 3, "max": 10}, # البحث في مدى بين 3 و 10
    },
}

# 2. خاص به ID للحصول على W&B على خوادم Sweep إنشاء الـ
sweep_id = wandb.sweep(sweep_config, project="ride-duration")

# 3. لتنفيذ 20 محاولة تدريب تلقائياً Agent تشغيل الـ
# باختبار القيم الأنسب لتجربتها بناء على النتائج السابقة W&B في كل محاولة، سيتم سيرفر
wandb.agent(sweep_id, function=train_fn, count=20)
```

لماذا الـ Sweeps في W&B قوية جداً؟

- **method: bayes (البحث البايزي)** : بدلاً من تجربة كل الاحتمالات يدوياً وهدر الوقت والمال (Grid Search)، يدرس الذكاء الاصطناعي نتائج المحاولات السابقة ليتوقع بذكاء أفضل قيم للمحاولة القادمة، مما يقلل عدد التجارب المطلوبة للوصول لأفضل دقة بشكل كبير.
- **الـ Agent** : هو عملية برمجية (Worker Process) تتصل بسيرفر W&B لتسحب التوليفة المقترحة التالية، تقوم بالتدريب، تسجل مقياس الأداء، ثم تكرر العملية دون تدخل بشري.

DVC (Data Version Control)، أو كما يُطلق عليه: "مُدير إصدارات البيانات والـ Models".

1. لماذا نحتاج DVC؟ (Why DVC)

برنامج **Git** مصمم في الأساس لمتابعة وإصدار ملفات النصوص والكود البرمجي. لكن في هندسة الذكاء الاصطناعي، نتعامل مع:

- ملفات بيانات خام (**CSV** أو **Parquet**) بحجم **500MB** أو أكثر.
- ملفات أوزان النماذج المدربة (**Weights** كـ **.pkl** أو **.pt**) بحجم **200MB** أو أكثر.

المشكلة الكبرى:

- موقع **GitHub** يمنع تماماً رفع أي ملف يتعدى حجمه **100MB**.
 - وحتى لو تم التغاضي عن ذلك، فإن محاولة حفظ ملفات البيانات الضخمة داخل **Git** تجعل تاريخ المشروع (**Git History**) ثقيل جداً وغير قابل للتحميل أو التصفح.
- الحل السحري مع DVC** : بدلاً من حفظ الملف الضخم نفسه في الـ **Git**، يقوم **DVC** وحيداً بحفظ الملف الفعلي في مخزن خارجي سحابي (مثل **AWS S3** أو **Google Cloud Storage**). وفي المقابل، ينشئ ملفاً صغيراً جداً كـ مؤشر (**Pointer**) بامتداد **.dvc**. يحتوي على كود تشفير فريد (**MD5 hash**) للملف الفعلي وحجمه.
- الـ **Git** : يتتبع ويحفظ ملف المؤشر الصغير هذا فقط.
 - الـ **DVC** : يتتبع ويجلب البيانات الفعلية من السحابة بناءً على هذا المؤشر.

وَلَا نَعْمِكُمْ ۖ فَإِذَا جَاءَتِ الطَّامَّةُ الْكُبْرَىٰ ۖ يَوْمَ يَتَذَكَّرُ الْإِنْسَانُ
مَا سَعَىٰ ۖ وَبُرُزَتِ الْجَحِيمُ لِمَن يَرَىٰ ۖ فَأَمَّا مَن طَغَىٰ ۖ وَءَاثَرَ
الْحَيَوَةَ الدُّنْيَا ۖ فَإِنَّ الْجَحِيمَ هِيَ الْمَأْوَىٰ ۖ وَأَمَّا مَن خَافَ
مَقَامَ رَبِّهِ ۖ وَنَهَى النَّفْسَ عَنِ الْهَوَىٰ ۖ فَإِنَّ الْجَنَّةَ هِيَ الْمَأْوَىٰ

﴿يوم يتذكر الإنسان ما سعى﴾
أعمال نُسيت ولكنها في كتاب أُحصيت،
فأعمل لنفسك ما يسرك أن تراه يوم القيامة..

عَالِمُ الْغَيْبِ وَالشَّهَادَةِ الْكَبِيرُ الْمُتَعَالِ (9) سَوَاءٌ مِنْكُمْ مَّنْ أَسْرَ الْقَوْلَ وَمَنْ جَهَرَ بِهِ وَمَنْ هُوَ مُسْتَخْفٍ بِاللَّيْلِ وَسَارِبٌ بِالنَّهَارِ (10) لَهُ مُعَقَّبَاتٌ مِّنْ بَيْنِ يَدَيْهِ
وَمِنْ خَلْفِهِ يَحْفَظُونَهُ مِنْ أَمْرِ اللَّهِ إِنَّ اللَّهَ لَا يُغَيِّرُ مَا بِقَوْمٍ حَتَّىٰ يُغَيِّرُوا مَا بِأَنْفُسِهِمْ ۖ وَإِذَا أَرَادَ اللَّهُ بِقَوْمٍ سُوءًا فَلَا مَرَدَّ لَهُ ۚ وَمَا لَهُمْ مِنْ دُونِهِ مِنْ آلٍ (11) هُوَ الَّذِي
يُرِيكُمْ الْبَرْقَ خَوْفًا وَطَمَعًا وَيُنشِئُ السَّحَابَ الثِّقَالَ (12) وَيُسَبِّحُ الرَّعْدُ بِحَمْدِهِ وَالْمَلَائِكَةُ مِنْ خِيفَتِهِ وَيُرْسِلُ الصَّوَاعِقَ فَيُصِيبُ بِهَا مَن يَشَاءُ وَهُمْ يُجَادِلُونَ فِي
اللَّهِ وَهُوَ شَدِيدُ الْمِحَالِ (13)

2. الخطوات الخمس لإعداد DVC (DVC Setup in 5 Commands)

لتشغيل الـ DVC وربطه بريبو الـ Git الخاص بك وتجهيز storage خارجي (سيرفر AWS S3 كمثال) ،
نستخدم الأوامر التالية في الـ Terminal:

1. تهيئة DVC داخل مجلد الـ Git الخاص بالمشروع

```
git init && dvc init
```

```
git add .dvc/ && git commit -m "init: add dvc"
```

2. تتبع ملف بيانات ضخمة (مثل ملف الرحلات raw csv)

```
dvc add data/raw/rides.csv
```

هذا الأمر يقوم تلقائياً بـ:

أ. إنشاء ملف مؤشر صغير اسمه data/raw/rides.csv.dvc

ب. إضافة مسار الملف الأصلي إلى ملف .gitignore. لمنع Git من تتبعه بالخطأ

```
git add data/raw/rides.csv.dvc data/raw/.gitignore
```

3. تعريف المخزن السحابي الخارجي وتحديد الإقليم الافتراضي (Default Remote)

```
dvc remote add -d s3remote s3://mlops-dvc-artifacts/data
```

```
dvc remote modify s3remote region us-east-1
```

4. رفع البيانات الفعلية للسحاب، ورفع كود الـ pointers للـ Git

```
dvc push && git push
```

5. على أي جهاز آخر للفريق (أو سيرفر الـ CI/CD) :

```
git clone <repo> && dvc pull
```

طرقه (قال : قال رسول الله صلى الله عليه وعلى آله وسلم : « الراحمون يرحمهم
الرحمن ارحموا من في الأرض يرحمكم من في السماء » .

3. خطوط الإنتاج القابلة لإعادة الإنتاج (DVC Pipelines - dvc.yaml)

لا يقتصر دور DVC على إدارة الملفات فقط، بل يمكنه أتمتة وإدارة خطوات العمل بأكملها في ملف معرف باسم `dvc.yaml`

الفكرة هنا هي أن **DVC** يتتبع المدخلات والمخرجات لكل مرحلة، وإذا قمت بكتابة أمر التشغيل (`dvc repro`) ، فلن يقوم بإعادة تشغيل أي خطوة إلا إذا تغيرت مدخلاتها بالفعل، مما يوفر ساعات طويلة من وقت الحساب!

هيكل ملف `dvc.yaml`:

```
stages:
  prepare:
    cmd: python src/prepare.py          # الأمر البرمجي للتشغيل
    deps:                               # المدخلات (Dependencies)
      - src/prepare.py
      - data/raw/rides.csv
    outs:                               # المخرجات (Outputs)
      - data/processed/train.parquet

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/processed/train.parquet
    params:                             # البارامترات المعتمد عليها التدريب من ملف
      - configs/config.yaml:
        - model.n_estimators
    outs:
      - models/rf_model.pkl
```

- **deps** : إذا تغير أي ملف في هذا القسم، سيعرف DVC تلقائياً أنه يجب إعادة تشغيل هذه المرحلة.
- **params** : قيم الـ Hyperparameters . لو قمت بتغيير الـ `n_estimators` في ملف الإعدادات، سيتعرف DVC على التغيير ويعيد تشغيل خطوة التدريب فقط دون خطوة تحضير البيانات.
- **outs** : المخرجات التي يقوم DVC بحفظها في الـ Cache تلقائياً.

عَنِ النَّبِيِّ صَلَّى اللَّهُ عَلَيْهِ وَسَلَّمَ فِيمَا رَوَى عَنِ اللَّهِ تَبَارَكَ وَتَعَالَى، أَنَّهُ قَالَ: يَا عِبَادِي، إِنِّي حَرَمْتُ الظُّلْمَ عَلَى نَفْسِي، وَجَعَلْتُهُ بَيْنَكُمْ مُحَرَّمًا، فَلَا تَظَالَمُوا، يَا عِبَادِي، كُلُّكُمْ ضَالٌّ إِلَّا مَنْ هَدَيْتُهُ، فَاسْتَهْدُونِي أَهْدِكُمْ، يَا عِبَادِي، كُلُّكُمْ جَائِعٌ إِلَّا مَنْ أَطْعَمْتُهُ، فَاسْتَطْعِمُونِي أَطْعَمَكُمْ، يَا عِبَادِي، كُلُّكُمْ عَارٍ إِلَّا مَنْ كَسَوْتُهُ، فَاسْتَكْسُونِي أَكْسَكُمْ، يَا عِبَادِي، إِنَّكُمْ تُخْطِئُونَ بِاللَّيْلِ وَالنَّهَارِ، وَأَنَا أَغْفِرُ الذُّنُوبَ جَمِيعًا، فَاسْتَغْفِرُونِي أَغْفِرْ لَكُمْ، يَا عِبَادِي، إِنَّكُمْ لَنْ تَبْلُغُوا ضُرِّي فَتُضْرُونِي، وَلَنْ تَبْلُغُوا نَفْعِي فَتَنْفَعُونِي، يَا عِبَادِي، لَوْ أَنَّ أَوَّلَكُمْ وَآخِرَكُمْ وَبَيْنَكُمْ كَانُوا عَلَى أَجْفَرِ قَلْبٍ رَجُلٍ وَاحِدٍ مَا نَقَصَ ذَلِكَ مِنْ مُلْكِي شَيْئًا، يَا عِبَادِي، لَوْ أَنَّ أَوَّلَكُمْ وَآخِرَكُمْ وَبَيْنَكُمْ كَانُوا فِي صَعِيدٍ وَاحِدٍ فَسَأَلُونِي، فَأَعْطَيْتُ كُلَّ إِنْسَانٍ مَسْأَلَتَهُ، مَا نَقَصَ ذَلِكَ مِمَّا عِنْدِي إِلَّا كَمَا يَنْقُصُ الْخَيْطُ إِذَا أُدْخِلَ الْبَحْرَ، يَا عِبَادِي، إِنَّمَا هِيَ أَعْمَالُكُمْ أَحْصِيهَا لَكُمْ، ثُمَّ أَوْفِيكُمْ بِهَا، فَمَنْ وَجَدَ خَيْرًا فَلْيَحْمَدِ اللَّهَ، وَمَنْ وَجَدَ غَيْرَ ذَلِكَ فَلَا يَلُومَنَّ إِلَّا نَفْسَهُ. وفي رواية: إِنِّي حَرَمْتُ عَلَى نَفْسِي الظُّلْمَ وَعَلَى عِبَادِي، فَلَا تَظَالَمُوا.

الراوي : أبو ذر الغفاري | المحدث : مسلم | المصدر : صحيح مسلم

الصفحة أو الرقم: 2577 | خلاصة حكم المحدث : [صحيح]

التخريج : من أفراد مسلم على البخاري

4. ربط نسخة الكود بنسخة البيانات (Data & Code Co-versioning)

باستخدام Git و DVC معاً، يمكنك السفر عبر الزمن والرجوع لأي تجربة سابقة ببياناتها وكودها بنسبة 100%

- لنفترض أننا نريد وسم النسخة الأولى التي تدربت على 500 ألف سطر

```
git tag -a v1.0 -m "baseline model"
```

```
git push origin v1.0
```

- كبر حجم البيانات لـ 1 مليون سطر، نقوم بتحديث التتبع والTag
○ `dvc add data/raw/rides.csv` سينتج hash جديد داخل ملف الـ `.dvc`.

```
git commit -am "data:expand to 1M rows"
```

```
git tag -a v2.0 -m "retrained on 1M rows"
```

```
git push origin v2.0 && dvc push
```

- الآن، للتنقل الفوري بين النسختين ببياناتهما الفعلية:

```
git checkout v1.0 && dvc checkout
```

ستقوم DVC بتبديل ملف الـ `csv` الفعلي على جهازك لنسخة الـ 500K

```
git checkout v2.0 && dvc checkout
```

ستقوم DVC بتبديله فوراً لنسخة الـ 1M

- `dvc status` : يشبه `git status` تماماً، يوضح لك ما هي الملفات المعدلة، المفقودة، أو التي لم يتم رفعها للمخزن الخارجي بعد.

- `dvc diff v1.0 v2.0` : يوضح لك الفروقات بدقة وحجم التغير في البيانات بين الوسمين (Auditable Data Lineage)

« يقال للولدان يوم القيامة: ادخلوا الجنة » قال : « فيقولون يارب حتى يدخل آباؤنا وأمهاتنا » قال : « فيأتون » قال « فيقول الله عز وجل : مالي أراهم محبطين^(١) ادخلوا الجنة » قال « فيقولون : يارب آباؤنا وأمهاتنا » قال « فيقول : ادخلوا الجنة أنعم وآباؤكم » . (حديث حسن)

5. تشغيل DVC داخل بيئة الـ CI/CD (GitHub Actions)

عند تشغيل اختبارات التدريب التلقائية داخل GitHub Actions، لا نريد كتابة بيانات الدخول السرية للـ Storages (كـ AWS Keys) داخل الكود، بل نمررها كـ **Secrets** مشفرة.

كما أننا نقوم بعمل **Partial pull (سحب جزئي)** لسحب الملفات التي يحتاجها التدريب فقط، فمن غير المنطقي جعل الـ Runner في الـ CI يقوم بتحميل باقية بيانات كاملة بحجم 40GB !

```
- name: Configure DVC remote credentials
  env:
    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
  run: |
    pip install "dvc[s3]"
    dvc pull data/processed/ # سحب مجلد البيانات المعالجة التي يحتاجها التدريب فقط

- name: Run the DVC pipeline
  run: dvc repro # تشغيل خط الإنتاج وتحديث ما لزم فقط
```

6. الربط بين DVC و MLflow (The Grand Lineage)

لتحقيق أقصى درجات الاحترافية وإعادة الإنتاجية بنسبة 100% للأبد، نقوم بربط الأدوات كالتالي:

1. **DVC** يتولى إصدار البيانات ومخرجاتها: (البيانات الخام، الميزات المهندسة، التقسيمات) ويتم الإشارة لها بـ git commit فريد (مثال: abc123f).

2. **MLflow** يتولى تتبع النموذج والـ Run: وحفظ الأوزان الناتجة والـ metrics عبر ID فريد للتجربة run_id

عند مراجعة إصدار النموذج model v3.2 في MLflow، ستجد أنه يشير لـ run_id = 9a3f8b2c والذي يشير بدوره لـ **dvc tag rides-v2.1** المرتبط بـ **git commit abc123f** والذي يمثل بالضبط تدريبه على M2.1 سطر بيانات من شهر يناير! هذا هو قمة الاحتراف الهندسي في تتبع تدفق البيانات والموديلات (Data Lineage)

- لَأَنْ أَقُولَ : سُبْحَانَ اللَّهِ وَالْحَمْدُ لِلَّهِ وَلَا إِلَهَ إِلَّا اللَّهُ وَاللَّهُ أَكْبَرُ أَحَبُّ إِلَيَّ مِمَّا طَلَعَتْ عَلَيْهِ الشَّمْسُ

ننتقل الآن إلى **Code Quality** ، وتحت هذا العنوان شعار هندسي صارم : **"Make bad code impossible to commit"** (اجعل الكود السيئ مستحيلاً حتى في الدخول للمشروع).

في فرق العمل الاحترافية، لا يمكننا الاعتماد على "نوايا المبرمجين الحسنة" أو المراجعة اليدوية للتأكد من نظافة الكود. نريد أتمتة كاملة تمنع أي كود غير منسق، أو يحتوي على أخطاء برمجية، من الوصول إلى مستودع المشروع (**Git Repository**) ، وذلك قبل أن يصل إلى مرحلة الاختبار والـ **CI/CD** .

سنتناول بالتفصيل الأدوات الثلاثة الأساسية وكيفية دمجها بالكامل في بيئة العمل.

1. الأدوات الثلاثة لإنهاء نقاشات "تنسيق الكود" للأبد (Style Debates)

بدلاً من تضيق وقت الفريق في مراجعة الكود (Code Reviews) على أشياء تافهة مثل "لماذا وضعت مسافة هنا؟" أو "لماذا لم ترتب المكتبات المستوردة؟"، نستخدم ثلاثة أدوات رئيسية تنهي هذا الجدل تلقائياً:

أولاً Ruff :

- ما هو؟ هو أداة فحص كود (Linter & Formatter) فائقة السرعة مكتوبة بلغة **Rust** .
- لماذا هو ثوري؟ في السابق، كنا نستخدم 3 أدوات منفصلة : (Flake8 للبحث عن الأخطاء، isort لترتيب الـ Imports ، و pylint للفحص العميق). **Ruff** جاء ليعوض كل هذه الأدوات مجتمعة، وبسرعة مذهلة تتراوح بين 10 إلى 100 ضعف سرعة الأدوات القديمة.
- أمر التشغيل اليدوي :

```
ruff check src/
```

ثانياً Black :

- ما هو؟ هو منسق كود (Opinionated Formatter) يتبع فلسفة "تنسيق واحد موحد للجميع" (Zero Configuration) .
- فلسفته: بغض النظر عن طريقة كتابتك للكود، بمجرد تشغيل **Black**، سيبدو ملفك متطابقاً تماماً مع ملفات باقي أعضاء الفريق، مما يسهل قراءة الأكواد داخل الشركة وكأن شخصاً واحداً هو من كتب المشروع بالكامل.
- أمر التشغيل اليدوي :

```
black src/
```

(ملاحظة: الإصدارات الحديثة من Ruff أصبحت تحتوي على أداة ruff-format التي تقوم بنفس وظيفة Black تماماً وبسرعة الـ Rust الرهيبة، وهي التي سنستخدمها في إعداداتنا).

ثالثاً pre-commit : (الحارس)

• ما هو؟ هو إطار عمل لإدارة الـ **Git Hooks** .

• كيف يعمل؟ الـ **Git Hook** هو سكريبت يعمل تلقائياً عند تنفيذ أمر معين في **Git** أداة **pre-commit**

تقوم بإنشاء حارس عند بوابة أمر **git commit** . عندما تكتب **git commit -m "add train logic"** ، يقوم الـ **pre-commit** بإيقافك وتشغيل **Ruff** و **Black** على الملفات المعدلة فقط .

• إذا كان الكود نظيفاً: يكتمل الـ **Commit** بنجاح.

• إذا كانت هناك أخطاء يمكن إصلاحها تلقائياً: يقوم الـ **pre-commit** بإصلاحها ويطلب منك مراجعتها وإعادة الـ **Commit** .

• إذا كان هناك خطأ قاتل: يتم إلغاء الـ **Commit** بالكامل ويُجبرك على إصلاح الخطأ أولاً.

2. الشرح التفصيلي لملف الإعدادات **pre-commit-config.yaml**.

لكي تعمل هذه المنظومة في مشروعك، نقوم بإنشاء ملف إعدادات في الجذر الرئيسي للمشروع (**Repo** **Root** باسم **pre-commit-config.yaml**).

repos:

```
# 1. لعمل الفحص والتنسيق Ruff ربط مستودع أداة
- repo: https://github.com/astreal-sh/ruff-pre-commit
  rev: v0.4.4 # تثبيت إصدار الأداة بدقة لتوحيد بيئة العمل لدى الفريق
  hooks:
    - id: ruff # لإصلاح المشاكل الآمنة تلقائياً Linter تشغيل الـ
      args: [--fix] # لإصلاح ما يمكن إصلاحه تلقائياً دون إزعاجك Ruff توجيهه
    - id: ruff-format # فائق السرعة Black البديل لـ Formatter تشغيل الـ

# 2. الاستعانة بالخطافات القياسية لحماية الكود من الهفوات البسيطة
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace # حذف أي مسافات زائدة في نهايات السطور تلقائياً
    - id: end-of-file-fixer # (معيار POSIX) التأكد من وجود سطر فارغ في نهاية كل ملف
    - id: check-yaml # (Kubernetes أو DVC لمنع انهيار إعدادات) YAML التأكد من صحة بناء ملفات الـ
    - id: check-merge-conflict # (Merge Conflicts) منع رفع كود يحتوي على تعارضات دمج معلقة!
```

• وعند مسلم (١٠٣٦) من حديث أبي أمامة رضي الله عنه قال : قال رسول الله صلى الله عليه وعلى آله وسلم : « يا ابن آدم إنك إن تبذل الفضل خير لك ، وإن تمسكه شر لك ولا تلام على كفاف ، وابدأ بمن تعول ، واليد العليا خير من اليد السفلى » .

ولمزيد في هذا الباب انظر رسالتنا ذم البخل .

3. ما وراء كواليس هذا الملف (Deep Technical Insights) :

- **تثبيت الإصدارات (rev) :** نقوم بتحديد رقم الإصدار بدقة (مثل v0.4.4) لضمان أن كل مطور في الفريق يستخدم نفس قواعد التنسيق بالضبط. لو تركناها عشوائية، قد يقوم مطور بتنسيق ملف، ثم يأتي مطور آخر بإصدار أحدث ليعيد تنسيقه فيحدث تضارب في مستندات الـ Git.
- **خيار المصلح التلقائي (args: [--fix]) :** يقوم Ruff بإصلاح الأخطاء البسيطة تلقائياً (مثل حذف المكتبات المستوردة غير المستخدمة Unused Imports أو تنظيم الترتيب الأبجدي للمكتبات). أما المشاكل المعقدة (مثل متغير تم استخدامه قبل تعريفه)، فيتوقف الـ Commit ويعرض لك مكان الخطأ والسطر لتعديله بنفسك.
- **الخطافات الصغيرة لها وزن ذهب (The Small Hooks Matter) :**
 - **check-merge-conflict :** هل حدث معك من قبل أن قمت برفع ملف كود يحتوي على علامات الـ Git المزعجة مثل HEAD <<<<<< و =====؟ هذه الهفوة تسبب ضوضاء هائلة أثناء مراجعة الكود وقد تمنع تشغيل السيرفر. هذا الـ Hook البسيط يقضي على هذه المشكلة تماماً.

4. دورة العمل الفعلية للمهندس (How it works in practice)

لتفعيل هذا النظام محلياً على جهازك، تسير الأمور كالتالي:

1. تثبيت الأداة بداخل المشروع:

```
pip install pre-commit
```

```
pre-commit install
```

(بمجرد كتابة pre-commit install، يتم ربط الأداة بملفات الـ Git الخفية في مجلد .git/hooks. لتصبح الحارس المعتمد محلياً).

2. التشغيل العملي أثناء العمل: أنت الآن كتبت كوداً جديداً غير مرتب وتريد عمل Commit :

```
git add src/train.py
```

```
git commit -m "feat: add lightgbm model"
```

تلقائياً في هذه اللحظة، ستظهر لك لوحة فحص سريعة في الـ Terminal :

```
Ruff.....Passed
Ruff-format.....Failed
- files mutated (formatting applied)
Trailing Whitespace.....Passed
Check YAML.....Passed
```

بما أن التنسيق فشل (Failed) وقامت الأداة بتعديل الملف وتنسيقه تلقائياً، سيتم إلغاء Commit . كل ما عليك فعله الآن هو:

```
git add src/train.py
```

 # لإضافة التعديلات والتحسينات التلقائية التي قامت بها الأداة

```
git commit -m "feat: add lightgbm model"
```

 # سيمر هذه المرة بنجاح تام!

5. علاقة Code Quality بباقي أجزاء Pipeline

أداة **pre-commit** تحميك محلياً (Locally) على جهازك قبل Commit . لكن كأمان إضافي، في بيئة الـ **CI/CD** (مثل GitHub Actions)، نقوم بإعادة تشغيل نفس الفحص البرمجي باستخدام **Ruff** كخطوة ثانية أساسية مباشرة بعد سحب الكود وقبل تشغيل أي اختبارات (pytest) . إذا استطاع شخص تخطي الـ **pre-commit** المحلي بطريقة ما (عبر كتابة `git commit --no-verify` مثلاً)، فإن سيرفر الـ **CI** سيكتشف ذلك ويفشل الـ Pipeline بالكامل لحماية الـ Production .

تعالاً ننتقل الآن إلى **Terraform (Infrastructure as Code - IaC)**، وتحت هذا العنوان مبدأ هندسي غاية في الأهمية: "اكتب كوداً، تحصل على بنية تحتية (Write code, get infrastructure)" . في بيئات العمل الاحترافية، لا يمكننا إعداد موارد السحاب (Cloud Resources) يدوياً. سنتعلم هنا كيف نتحكم في البنية التحتية بالكامل ككود برمجي تتبع إصداراته عبر Git .

1. المقارنة: بدون Terraform مقابل استخدام Terraform

استخدام Terraform (ClickOps)	استخدام Terraform (IaC)
تقوم بالنقر يدوياً داخل لوحة تحكم AWS أو GCP	البنية التحتية بالكامل مكتوبة ومحددة داخل ملفات .tf
لا يوجد أي سجل يوثق ما قمت بإنشائه أو لماذا قمت بذلك.	يتم تتبع الملفات بالكامل عبر Git مع ميزة مراجعة التغييرات وتاريخها.
لا يمكن للفريق إعادة إنتاج بيئة مطابقة لبيئتك.	كود واحد يبني نفس البيئة المتطابقة في كل مرة لجميع المطورين.
في حال حدوث كارثة أو انهيار للنظام، تبدأ البناء يدوياً من الصفر.	استعادة النظام من الكارثة تعني فقط تشغيل <code>git clone</code> ثم <code>terraform apply</code> .

2. المصطلحات الأربعة الأساسية في Terraform التي يجب حفظها (Core Concepts)

لفهم كيفية عمل Terraform ، هناك أربعة مصطلحات أساسية يقوم عليها النظام بالكامل:

1. **Provider (المزود) :** هو عبارة عن إضافة برمجية (Plugin) تمكن Terraform من التحدث مع البيئة السحابية المستهدفة (مثل AWS ، GCP ، Azure ، أو حتى Docker). يتم تحميل هذه الإضافة تلقائيًا بمجرد كتابة الأمر `terraform init` .

2. **Resource (المورد) :** يمثل قطعة بنية تحتية واحدة تريد إنشاءها. قد تكون هذه القطعة عبارة عن **S3 bucket** لتخزين البيانات، أو سيرفر **EC2 instance** ، أو سجل نطاقات **DNS record**

3. **State (ملف الحالة-terraform.tfstate) :** هو الخريطة أو قاعدة البيانات المحلية التي توضح لـ Terraform العلاقة الحالية بين الكود المكتوب على جهازك والموارد الفعلية الموجودة بالفعل على السحابة.

○ **تحذير هندسي :** لا تقم أبدًا برفع ملف **terraform.tfstate** إلى **Git** لأنه يحتوي على بيانات سرية وحساسة مثل كلمات المرور وعناوين خوادم البيانات. في فرق العمل، نستخدم **S3 backend** لتخزين هذا الملف سحابيًا بشكل آمن ومشارك لضمان تزامن العمل بين الجميع ومنع التعديل المتزامن (State Locking) .

4. Plan and Apply (التخطيط والتنفيذ) :

○ **plan :** يعرض لك معاينة سريعة وقراءة فقط للتغييرات التي ستحدث في السحابة قبل تنفيذها فعلياً.

○ **apply :** هو الأمر الفعلي الذي يقوم بطلب وإنشاء أو تحديث الموارد على السحابة بناءً على خطة الـ plan المعتمدة.

3. ما هي البنية التحتية التي يحتاجها مشروع الـ MLOps الخاص بنا؟

لكي يعمل نظام الذكاء الاصطناعي الخاص بنا بكفاءة في بيئة الإنتاج، يحتاج Terraform لإنشاء 6 قطع أساسية من البنية التحتية:

1. **S3 bucket (مخزن السحاب) :** ليعمل كمخزن بعيد (DVC Remote) لحفظ ملفات البيانات والنماذج المدربة.

2. **MLflow server :** سيرفر لتعقب التجارب وحفظ أوزان النماذج والـ Model Registry.

3. **Docker registry (مثل Amazon ECR):** لدفع وحفظ حاويات Docker الناتجة عن عمليات الـ CI/CD

4. **Secrets manager :** لحفظ مفاتيح الـ API وكلمات المرور مشفرة دون كتابتها في الكود.

5. **Compute VM (مثل EC2):** لتوفير القوة الحسابية اللازمة لتشغيل مهام التدريب والـ Training Jobs

6. **IAM roles :** لتحديد صلاحيات الوصول بدقة (من يستطيع القراءة من الـ S3 ومن يمكنه تشغيل الـ VM).

بدون الـ IaC ، يحتاج أي مهندس جديد ينضم للفريق إلى ساعتين على الأقل من الشرح والمساعدة لتجهيز بيئته يدوياً. أما باستخدام الـ IaC ، يكتب المهندس **git clone** ثم **terraform apply** ليحصل في 5 دقائق على بيئة عمل حقيقية مطابقة تماماً للـ Production.

4. كيف نكتب كود الـ Terraform؟ (infra/main.tf)

تعتمد كتابة كود Terraform على استعارة القطع البرمجية الجاهزة من الـ Documentation على موقع **registry.terraform.io** ثم ملء قيمك الخاصة بها.

إليك نموذج عملي مبسط لإنشاء **S3 bucket** مخصص لمشروعنا ليعمل كمخزن لـ DVC :

```
# إصداره Provider إعداد المتطلبات وتحديد الـ 1.
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"      # تثبيت الإصدار الرئيسي لضمان عدم انهيار الكود مستقبلاً
    }
  }
}

# تحديد إقليم السحابة للعمل 2.
provider "aws" {
  region = "us-east-1"      # الإقليم الجغرافي الذي ستنشأ فيه الموارد
}

# DVC لتخزين ملفات S3 Bucket المورد الفعلي لإنشاء 3.
resource "aws_s3_bucket" "dvc" {
  bucket = "mlops-dvc-artifacts-ride-duration" # اسم المخزن (يجب أن يكون فريداً عالمياً)
}
```

• **تثبيت الإصدار (5.0 >~):** يضمن أن الكود سيعمل مع أي إصدار فرعي من عائلة 5.x ولكنه يمنع الانتقال التلقائي إلى الإصدار 6.0 الذي قد يحتوي على تغييرات جذرية تعطل عمل البنية التحتية الخاصة بك.

• **الصلاحيات ومفاتيح الدخول:** لا نكتب مفاتيح AWS السرية داخل ملفات **tf**. مطلقاً. يعتمد Terraform تلقائياً على إعدادات AWS CLI الخاصة بجهازك محلياً أو المتغيرات البيئية المحقونة في الـ CI.

5. دورة العمل والأوامر السبعة الأساسية (The 7 Commands Workflow)

لكتابة وتنفيذ أي تغيير في البنية التحتية، نسير وفق دورة العمل الهندسية التالية عبر هذه الأوامر:

1. **terraform init:** لتحميل الـ Providers وتهيئة الـ Backend.
2. **terraform fmt:** لتنسيق ملفات الكود تلقائياً وجعلها مطابقة للمعايير القياسية لـ HashiCorp.
3. **terraform validate:** للتحقق من صحة القواعد البرمجية (Syntax) دون الاتصال بالسحابة.
4. **terraform plan -out=tfplan:** لمعاينة التغييرات القادمة وحفظ الخطة في ملف مؤقت يسمى tfplan لضمان عدم تغير الحالة أثناء المراجعة.
5. **terraform apply tfplan:** لتطبيق الخطة المحفوظة فعلياً وإنشاء الموارد.
6. **terraform output ecr_repo_url:** لقراءة المخرجات وعناوين الخوادم التي تم إنشاؤها لاستخدامها في كود الـ API أو الـ CI/CD.
7. **terraform destroy:** لهدم وإزالة كافة الموارد التي تم إنشاؤها بالكامل بضغط زر واحدة لتوفير التكلفة عند انتهاء العمل.

عَنْ عَبْدِ اللَّهِ بْنِ مَسْعُودٍ رَضِيَ اللَّهُ عَنْهُ قَالَ: قَالَ رَسُولُ اللَّهِ صَلَّى اللَّهُ عَلَيْهِ وَسَلَّمَ: ((إِنِّي لأَعْلَمُ آخِرَ أَهْلِ النَّارِ خُرُوجًا مِنْهَا، وَآخِرَ أَهْلِ الْجَنَّةِ دُخُولًا الْجَنَّةَ: رَجُلٌ يَخْرُجُ مِنَ النَّارِ حَبْوًا، فَيَقُولُ اللَّهُ تَبَارَكَ وَتَعَالَى لَهُ: اذْهَبْ فَادْخُلِ الْجَنَّةَ، فَيَأْتِيهَا فَيُخَيَّلُ إِلَيْهِ أَنَّهَا مَلَأَى، فَيَرْجِعُ فَيَقُولُ: يَا رَبِّ، وَجَدْتُهَا مَلَأَى، فَيَقُولُ اللَّهُ تَبَارَكَ وَتَعَالَى لَهُ: اذْهَبْ فَادْخُلِ الْجَنَّةَ، قَالَ: فَيَأْتِيهَا فَيُخَيَّلُ إِلَيْهِ أَنَّهَا مَلَأَى، فَيَرْجِعُ فَيَقُولُ: يَا رَبِّ، وَجَدْتُهَا مَلَأَى، فَيَقُولُ اللَّهُ تَعَالَى لَهُ: اذْهَبْ فَادْخُلِ الْجَنَّةَ؛ فَإِنَّ لَكَ مِثْلَ الدُّنْيَا وَعَشْرَةَ أَمْثَالِهَا - أَوْ إِنَّ لَكَ عَشْرَةَ أَمْثَالِ الدُّنْيَا - قَالَ: فَيَقُولُ: أُنْسَخِرُ بِي - أَوْ أُنْضَحُّ بِي - وَأَنْتَ الْمَلِكُ؟ قَالَ: لَقَدْ رَأَيْتُ رَسُولَ اللَّهِ صَلَّى اللَّهُ عَلَيْهِ وَسَلَّمَ ضَحَكَ حَتَّى بَدَتْ نَوَاجِذُهُ، قَالَ: فَكَانَ يُقَالُ: ذَاكَ أَدْنَى أَهْلِ الْجَنَّةِ مَنْزِلَةً))

ننتقل الآن إلى **CI/CD with GitHub Actions** . وتحت هذا العنوان، الشعار الهندسي للمرحلة هو :
"أتمتة كل شيء على كل عملية Push". (Automate everything on every push).
في هذا الجزء، سنتعلم كيف نربط كل الأجزاء التي بنيناها سابقاً (من فحص جودة الكود، والبيانات، والاختبارات) داخل pipeline متكامل يعمل تلقائياً بمجرد رفع الكود على منصة **GitHub** .

1. ما هو الـ CI والـ CD؟ (The Golden Definitions)

ينقسم هذا المفهوم إلى شقين أساسيين يعملان معاً:

أولاً: الـ CI (Continuous Integration - الدمج المستمر)

يهدف إلى التحقق التلقائي من صحة وأمان الكود الجديد قبل دمجها مع الفرع الرئيسي للمشروع (main) ويتضمن:

1. **Lint and format check** : تشغيل أداة Ruff للتأكد من نظافة الكود وتنسيقه.
2. **Unit & integration tests** : تشغيل اختبارات pytest .
3. **Coverage gate** : وضع بوابة جودة تمنع تمرير الكود إذا كانت نسبة تغطية الاختبارات أقل من 80%
4. **Build verification** : التأكد من أن الكود الجديد لا يسبب أي مشكلة في تشغيل بيئة العمل.

ثانياً: الـ CD (Continuous Delivery/Deployment - التسليم/النشر المستمر)

يهدف إلى تجهيز كود الموديل والـ API المعتمد ونشره في بيئة الإنتاج:

1. **Docker image built** : بناء حاوية Docker تحتوي على الكود والاعتماديات.
2. **Image pushed to registry** : رفع الحاوية إلى مخزن الحاويات (مثل Docker Hub أو AWS ECR).
3. **Versioned by Git SHA** : وسم الحاوية برمز التشفير الفريد الخاص بالـ Commit لضمان سهولة الرجوع إليها وتتبعها.
4. **Automatic deploy** : إعادة تشغيل الخدمة تلقائياً في الـ Production باستخدام الحاوية الجديدة.

حدثنا عبد الله بن مسلمة عن مالك عن ابن شهاب عن أبي سلمة وأبي عبد الله الأغر عن أبي هريرة رضي الله عنه أن رسول الله صلى الله عليه وعلى آله وسلم قال :
« ينزل ربنا تبارك وتعالى كل ليلة إلى السماء الدنيا حين يبقى ثلث الليل الآخر يقول : من يدعوني فأستجيب له ، من يسألني فأعطيه ، من يستغفرني فأغفر له » .
(حديث صحيح)

2. محركات إطلاق خط الإنتاج (Workflow Triggers)

في بيئة البرمجيات العادية، يتم إطلاق الـ Pipeline عند تعديل الكود فقط. أما في أنظمة الـ MLOps، نحتاج إلى أربعة Triggers مختلفة لتشغيل خط الإنتاج:

```
on:
  push:
    branches: [main]          # 1. مباشر لفرع Push عند عمل (للمطورين)
  pull_request:
    branches: [main]
    paths:
      - "src/**"              # 2. Pipeline الفلتر بالمسار: لا يتم بتشغيل الـ
      - "train.py"            ! فقط README كان التعديل في ملف الـ

  schedule:
    - cron: "0 2 * * 1"       # 3. كل يوم اثنين الساعة 2:00 صباحاً
      !إعادة تدريب النموذج ببيانات جديدة تلقائياً (UTC بتوقيت

  workflow_dispatch:          # 4. زر تشغيل يدوي تفاعلي داخل واجهة
    !للطوارئ
    inputs:
      reason:
        description: "Why are you running this?"
        required: true
```

ملاحظة تخص الـ MLOps :

- **Drift signal trigger (إشارة الانحراف):** في الأنظمة المتطورة، إذا اكتشف نظام المراقبة أن هناك انحرافاً في البيانات (**Data Drift**) بقيمة أعلى من المعيار المحدد (مثل مقياس $PSI > 0.25$)، يرسل السيرفر إشارة تلقائية تسمى repository_dispatch ليقوم GitHub Actions ببدء تدريب إسعافي فوراً للموديل.

3. الشرح التفصيلي لملف خط الإنتاج الكلاسيكي (.github/workflows/ci.yml)

يتم كتابة كود خط الإنتاج في ملف YAML يوضع بداخل المجلد الخاص بالـ workflows . إليك تفصيل مراحل التشغيل:

المرحلة الأولى: الفحص والاختبار (Lint and Test Job)

```
jobs:
  lint-and-test:
    runs-on: ubuntu-latest      # GitHub تشغيل الخطوات على سيرفر لينكس نظيف تماماً ومجانى تمجه لك
    steps:
      - uses: actions/checkout@v4 # 1. سحب كود المشروع من مستودعك
      - uses: actions/setup-python@v5 # 2. إصدارها Python تجهيز له
        with:
          python-version: "3.11"
      - uses: actions/cache@v4     # 3. في المرات القادمة بدلاً من تحميلها من الصفر Pip تسريع سحب المكتبات من الـ
        with:                      Cache ميرة الـ
          path: ~/.cache/pip
          key: "${{ runner.os }}-pip
      - run: pip install -e .[dev] # 4. تثبيت المشروع والمكتبات الخاصة بالتطوير
      - run: ruff check src/        # 5. لفحص جودة الكود تلقائياً تشغيل Ruff
      - run: pytest --cov=src --cov-fail-under=80 -v # 6. %تشغيل الاختبارات ورفض التمرير لو النغطية أقل من 80
```

- **uses** مقابل **run** : الكلمة المفتاحية **uses** تسحب أداة جاهزة ومطورة مسبقاً من متجر GitHub (Marketplace)، بينما **run** تقوم بتشغيل أوامر كلاسيكية في الـ Terminal.
- **--cov-fail-under=80** : هي حارس الجودة البرمجية، إذا أهمل المطور كتابة الاختبارات للكود الجديد وهبطت النسبة، سيفشل الـ Pipeline فوراً ولن يُسمح بدمج الكود.

المرحلة الثانية: بناء حاوية Docker والرفع للـ Registry (Build & Push Job)

هذه الخطوة لا يجب أن تعمل إلا إذا نجحت المرحلة الأولى تماماً لضمان عدم تعبئة الـ Registry بحاويات تالفة:

```
build-and-push:
  runs-on: ubuntu-latest
  needs: lint-and-test # تماماً lint-and-test إلا لو نجح الـ Job الاعتمادية: لا تبدأ هذا الـ
  if: github.ref == 'refs/heads/main' # main لا نرفع الحاوية للإنتاج إلا لو تم الدمج فعلياً في فرع
  steps:
    - uses: actions/checkout@v4

    # الأمانة GitHub مشفرة ومحفوظة في إعدادات Secrets باستخدام Docker Hub تسجيل الدخول لموقع
    - uses: docker/login-action@v3
      with:
        username: ${ secrets.DOCKERHUB_USERNAME }
        password: ${ secrets.DOCKERHUB_TOKEN }

    # latest: بكلمة Tag القصير و Git SHA بالـ Tag: تلقائية للحاوية (Tags) توليد وسوم
    - uses: docker/metadata-action@v5
      id: meta
      with:
        images: ${ secrets.DOCKERHUB_USERNAME }/ride-api
        tags: |
          type=sha,prefix=,format=short
          type=raw,value=latest

    # الـ Registry البناء والدفع للـ
    - uses: docker/build-push-action@v5
      with:
        context: .
        push: true
        tags: ${ meta.outputs.tags }
```

- **الأمان أولاً (secrets)** : لا نكتب كلمات المرور في الكود البرمجي أبداً. يتم تسجيلها في واجهة GitHub كـ **Encrypted Secrets**، ويقوم محرك Actions بحفظها وتشفيرها داخل السجلات (Logs) تلقائياً لمنع أي تسريب.
- **الـ Metadata والـ Git SHA** : عند تسمية حاوية الـ Docker بالـ SHA الخاص بالـ Commit، يصبح من السهل معرفة الكود الدقيق الذي بُنيت عليه هذه الحاوية وسهولة استرجاعه عند الرغبة في التراجع (Rollback).

4. الميزات المتقدمة: الـ Matrix والـ Layer Cache

1. الـ **Matrix Testing** (اختبار التوافقية) : إذا كنت تبني مكتبة برمجية وتريد التأكد من أنها تعمل على إصدارات مختلفة من بايثون في نفس الوقت، يمكنك استخدام ميزة الـ matrix :

```
strategy:
  matrix:
    python-version: ["3.10", "3.11", "3.12"] # سيتم إطلاق 3 خوادم برمجية متوازية لاختبار الكود عليها معاً
    fail-fast: false # لو فشل اختبار بايثون 3.10، استمر في فحص باقي الإصدارات لتظهر لك الصورة كاملة
```

2. الـ **Layer Cache** (تسريع بناء Docker) : تعتمد ميزة بناء الـ Docker على سحب الطبقات (Layers) السابقة المخزنة مؤقتاً من آخر بناء ناجح (latest) . هذا يختصر وقت البناء داخل الـ CI/CD من 3 دقائق إلى أقل من 30 ثانية!

5. نذفق الرحلة بالكامل بضغطة زر واحدة (The Entire Automation Flow)

عندما تضغط على لوحة المفاتيح وتكتب `git push origin main`، هذا ما يحدث خلف الكواليس في غضون 4 إلى 6 دقائق بدون أي تدخل بشري منك:

1. **Ruff + Black** : فحص فوري لجودة وتنسيق الملفات المرفوعة.
2. **pytest** : تشغيل الاختبارات والتحقق من بوابة الجودة للتغطية البرمجية.
3. **docker build** : تجميع كود الـ API والنموذج في حاوية نظيفة ومحسنة.
4. **docker push** : رفع الحاوية بوسوم التتبع الفريدة للـ Registry.
5. **terraform apply** : التحقق من ثبات البنية التحتية وتطبيق أي تعديل طرأ عليها.
6. **Deploy** : سحب الحاوية الجديدة على سيرفر الإنتاج الفعلي، وإعادة تشغيل الخدمة، وتشغيل الـ Health check للتأكد من استقرار السيرفر.

حدثنا أبو عامر ثنا هشام بن سعد عن زيد بن أسلم عن عطاء بن يسار عن أبي واقد الليثي قال :
« كُنَّا نَأْتِي النَّبِيَّ صَلَّى اللَّهُ عَلَيْهِ وَعَلَى آلِهِ وَسَلَّمَ إِذَا أُنْزِلَ عَلَيْهِ فَيُحَدِّثُنَا فَقَالَ لَنَا ذَاتَ يَوْمٍ : « إِنْ اللَّهُ عَزَّ وَجَلَّ قَالَ : إِنَّا أَنْزَلْنَا الْمَالَ لِإِقَامِ الصَّلَاةِ وَإِيتَاءِ الزَّكَاةِ وَلَوْ كَانَ لابنِ آدَمَ وَادٍ أَحَبُّ أَنْ يَكُونَ إِلَيْهِ ثَانٌ وَلَوْ كَانَ لَهُ وَادِيَانِ أَحَبُّ أَنْ يَكُونَ إِلَيْهِمَا ثَالِثٌ وَلَا يَمْلَأُ جَوْفَ ابْنِ آدَمَ إِلَّا التُّرَابُ ثُمَّ يَتَوَبُّ اللَّهُ عَلَى مَنْ تَابَ » .
(حديث حسن)

Integration Testing

في مرحلة الـ **Unit Testing**، قمنا باختبار كل دالة وكل ملف كود برمجى على حدة بمعزل عن باقي أجزاء النظام. أما في الـ **Integration Testing**، فنحن نقوم باختبار النظام كحلقة متصلة؛ لنتأكد من أن جميع المكونات (كود الـ API، معالجة البيانات، التحقق من المدخلات، ونموذج الذكاء الاصطناعي) تعمل معاً بتناغم تام دون أي انهيار عند استقبال طلبات حقيقية من المستخدمين.

1. كتابة كود اختبار التكامل الأساسي (tests/test_integration.py)

بدلاً من اختبار الدوال الداخلية، نقوم بمحاكاة خادم حقيقي (**TestClient**) يستقبل طلبات شبكة فعلية (HTTP Requests) ويمر عبر كامل طبقات نظامنا:

```
import pytest
from litestar.testing import TestClient
from unittest.mock import MagicMock
from src.api import app

# 1. نطاق ممتد على مستوى الملف بالكامل Fixture إعداد الـ
@pytest.fixture(scope="module")
def client():
    # 2. الفعلي دون استهلاك الذاكرة Model إنشاء نموذج وهمي يحاكي سلوك الـ
    mock = MagicMock(spec=RideDurationModel)
    mock.predict.return_value = 15.5

    # 3. API حقن النموذج الوهمي داخل إعدادات خادم الـ
    app.dependency_overrides[RideDurationModel] = lambda: mock

    # 4. تشغيل خادم الاختبارات وإرساله للبيئة
    with TestClient(app=app) as c:
        yield c

# 5. اختبار نجاح عملية التنبؤ بالكامل
def test_predict_success(client):
    r = client.post("/predict",
                    json={"distance_km": 5.0, "passengers": 2})
    assert r.status_code == 201
    assert "duration_min" in r.json()
```

أسرار هندسية من قلب هذا الكود:

- **scope="module"** (توفير الوقت والموارد): بشكل افتراضي، يقوم pytest بإعادة تشغيل الـ Fixtures مع كل دالة اختبار (مرة تلو الأخرى). تحديد النطاق كـ module يعني إنشاء خادم الاختبار والـ Mock لمرة واحدة فقط طوال تشغيل هذا الملف، مما يمنع إعادة بناء الخادم 20 مرة ويوفر وقتاً ثميناً جداً في خط الـ CI/CD

- **dependency_overrides (عزل النموذج عن اختبار الـ API):** نحن هنا نختبر مسارات الـ API والـ Serialization والـ Middleware ، وليس دقة النموذج الرياضي نفسه. لذا نقوم باستبدال النموذج الفعلي (الذي قد يستغرق ثوانٍ للتحميل ويستهلك مساحة من الذاكرة العشوائية) بنموذج وهمي سريع (**Mock**) يعيد فوراً القيمة 15.5
- **Real routing (الاختبار الحقيقي):** على الرغم من استخدام الـ Mock ، إلا أن الكود يمر عبر نظام التوجيه الفعلي للـ API ، وطبقة التحقق من البيانات (Pydantic) ، وطبقات الحماية وسجلات الـ Logs ، وهو ما يجعل هذا الاختبار اختبار تكامل حقيقي.

2. التحقق من المدخلات السيئة وحماية العقد برمجياً (Validation & Contract Tests)

لا يكفي اختبار السيناريو المثالي للنجاح (**Happy Path**) . المطور المحترف يركز على كيفية تصرف الخادم عند استقبال طلبات تالفة أو خبيثة للتأكد من أنه يرفضها بالرمز المناسب (**422 Unprocessable Entity**):

```
# اختبار النصفية والفحص التلقائي لرفض الطلبات التالفة
@pytest.mark.parametrize("payload,expected", [
    ({ "distance_km": -1.0, "passengers": 2 }, 422), # مسافة سالبة (مستحيل منطقياً)
    ({ "distance_km": 5.0, "passengers": 0 }, 422), # عدد ركاب صفر
    ({ "passengers": 2 }, 422), # حقل المسافة مفقود تماماً
    ({}), 422), # جسم الطلب فارغ بالكامل
])
def test_predict_validation(client, payload, expected):
    assert client.post("/predict", json=payload).status_code == expected
```

مفاهيم متقدمة لحماية النظام:

- **@pytest.mark.parametrize (الاختبار المتغير):** تتيح لك كتابة دالة اختبار واحدة وتمرير مصفوفة من الحالات المتعددة إليها. هذا يضمن كتابة كود مدمج ونظيف، مع إظهار رسالة خطأ مستقلة وواضحة لكل حالة تفشل بشكل منفصل.
- **test_health (نبض النظام الحرج):**

```
def test_health(client):
    assert client.get("/health").json()["status"] == "healthy"
```

أنظمة النشر والـ Load Balancers (مثل AWS ECS أو Kubernetes) تعتمد بالكامل على ضرب مسار /health للتأكد من أن الحاوية حية ومستقرة. أي خطأ غير مقصود يعطل هذا المسار سيتسبب في مقتل Deployment بأكمله بصمت؛ لذا تتبع هذا المسار واختباره أمر حتمي لا غنى عنه.

• الـ Contract test (اختبار عقد البيانات) :

```
def test_response_schema(client):  
    body = client.post("/predict", json={"distance_km": 5.0}).json()  
    assert set(body.keys()) == {"duration_min", "status"}
```

هذا الاختبار يقارن أسماء المفاتيح (Keys) العائدة من السيرفر. إذا قام مطور آخر بتعديل الكود وحذف حقل مثل status أو غير اسمه، سيكتشف اختبار التكامل هذا الأمر فوراً ويفشل. هذا يمنع حدوث ما يسمى بـ **Breaking Changes** التي قد تتسبب في انهيار تطبيقات الموبايل أو واجهات الويب التي تستهلك الـ API الخاص بك في الإنتاج.

قال رسول الله -صلي الله عليه وسلم- :

-والذي نَفْسِي بِيَدِهِ مَا أَحَدُكُمْ بِأَشَدَّ مُنَاشِدَةً فِي الْحَقِّ يَرَاهُ مُصِيبًا لَهُ مِنَ الْمُؤْمِنِينَ فِي إِخْوَانِهِمْ ، إِذَا رَأَوْا أَنْ قَدْ خُلِّصُوا مِنَ النَّارِ ، يَقُولُونَ : أَيُّ رَبَّنَا ! إِخْوَانُنَا كَانُوا يُصَلُّونَ مَعَنَا ، وَيَصُومُونَ مَعَنَا وَيُحْجُونَ مَعَنَا ، وَيُجَاهِدُونَ مَعَنَا ، قَدْ أَخَذْتَهُمُ النَّارُ ، فيقولُ اللهُ لَهُمْ : اذْهَبُوا فَمَنْ عَرَفْتُمْ صُورَتَهُ فَأَخْرِجُوهُ ، وَيُحَرِّمُ صُورَتَهُمْ عَلَى النَّارِ ، فيجدون الرجلَ قد أخذته النارُ إلى أنصافِ ساقيه ، وإلى ركبتيه ، وإلى حُقُوبِهِ ، فيُخْرِجونَ مِنْهَا بَشَرًا كَثِيرًا ، ثم يعودون فيتكلمون ، فيقولُ : اذْهَبُوا فَمَنْ وَجَدْتُمْ فِي قَلْبِهِ مِثْقَالَ قِيرَاطٍ خَيْرٍ فَأَخْرِجُوهُ ! فيُخْرِجونَ مِنْهَا بَشَرًا كَثِيرًا ، ثم يعودون فيتكلمون ، فلا يزالُ يقولُ لَهُمْ ذَلِكَ ، حتى يقولُ : اذْهَبُوا فَمَنْ وَجَدْتُمْ فِي قَلْبِهِ مِثْقَالَ ذَرَّةٍ فَأَخْرِجُوهُ ! فكان أبو سعيدٍ إِذَا حَدَّثَ بِهَذَا الْحَدِيثِ قَالَ : إِنْ لَمْ تُصَدِّقُوا فَاقْرَءُوا : ؟ إِنَّ اللَّهَ لَا يَظْلِمُ مِثْقَالَ ذَرَّةٍ وَإِنْ تَكُ حَسَنَةً يُضَاعِفْهَا وَيُؤْتِ مِنْ لَدُنْهُ أَجْرًا عَظِيمًا؟ فيقولون : رَبَّنَا لَمْ نَذَرْ فِيهَا خَيْرًا

خلاصة حكم المحدث : صحيح

الراوي : أبو سعيد الخدري | المحدث : ابن جرير الطبري | المصدر : تفسير الطبري | الصفحة أو الرقم :

4/1/121

سبحان الله كانوا يُصَلُّونَ ، وَيَصُومُونَ وَيُحْجُونَ ، وَيُجَاهِدُونَ ومع ذلك دخلوا ونحن إذا صلينا قمنا للصلاة كسالي ومع ذلك نأمن من مكر الله